GLA UNIVERSITY, GREATER NOIDA

Greater Noida – 201310, Uttar Pradesh, India

Title: Calories Burn Prediction System

SOFTWARE REQUIREMENT SPECIFICATION

Version 1.0

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE & ENGINEERING - AI

Submitted By

G. Chaitanya Mohan

Roll No.: 22515500034 | Section: ND-L1

Under the Supervision of	Reviewed By
Palash Raghuwanshi	
GLA University, Greater Noida	GLA University, Greater Noida

SCHOOL / DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING / APPLICATIONS

GLA UNIVERSITY, GREATER NOIDA – 201310

September 2026

Software Requirements Specification

Calories Burn Prediction System v1.0

Revision History

Version	Date	Change	Reason
1.0	04-09-2026	Initial version	First submission

1. Introduction

1.1 Purpose

This document specifies the requirements of the Calories Burn Prediction System v1.0, a supervised machine learning–based web application that predicts the number of calories a user burns during a workout session. It serves as the baseline against which implementation and test cases will be verified.

1.2 Scope

The system allows a registered user to log in, view their previous workout history, enter today's workout and physiological details (age, gender, height, weight, workout type, duration, heart rate, body temperature), and receive a predicted calories-burned value from a trained machine learning model. The prediction and workout details are saved to the user's history so the dashboard always reflects the latest activity. The system does not track real-time biometric sensor data and does not replace medical or fitness professional advice.

1.3 Definitions

Term	Definition
BMR	Basal Metabolic Rate — baseline calories the body burns at rest.
Feature vector	The set of numeric inputs (age, height, weight, duration, heart rate, body temperature, etc.) fed into the ML model.
Top prediction	The single calories-burned value returned by the regression model for a given workout.
History record	A saved entry containing a past workout's inputs and its predicted calories.

2. Overall Description

2.1 User Classes

User Class	Who They Are	What They May Do
Registered user	End-user of the fitness app	Register/log in, view workout history, enter today's workout, view prediction, have data

User Class	Who They Are	What They May Do
		saved automatically

2.2 Assumptions and Dependencies

- Depends on a public calories/exercise dataset — the Kaggle "Calories Burnt Prediction" dataset (calories.csv merged with exercise.csv on User_ID). See Section 3 for the exact source and mirror used.
- Depends on a web framework (Flask) for the UI, chosen for its lightweight routing and easy integration with a scikit-learn/XGBoost inference layer.
- Relies on scikit-learn and XGBoost for the regression model (Linear Regression as the interpretable baseline, XGBoost as the production model).
- Depends on a relational database (MySQL) for storing users and workout history, accessed via SQLAlchemy.

2.3 Constraints

- C1: Dataset does not contain continuous real-time sensor data (e.g. live heart-rate streaming) — inputs are entered manually per session.
- C2: Limited to supervised regression on tabular physiological and workout features.

3. Data Source and Baseline

Item	Value
Dataset name	Calories Burnt Prediction dataset (exercise.csv + calories.csv, joined on User_ID)
Source and URL	Originally published on Kaggle as "Calories Burnt Prediction". Mirror copies used for development: calories.csv and exercise.csv (linked below). Exact Kaggle listing URL to be pinned in the project README before final submission.
Licence / terms of use	Kaggle dataset licence listed as "Other" (permissive for academic/research use). The specific terms must be re-checked on the Kaggle dataset page and quoted verbatim in the final report before any deployment beyond coursework.
Number of rows and columns	15,000 rows × 9 columns after the exercise/calories merge: User_ID, Gender, Age, Height, Weight, Duration, Heart_Rate, Body_Temp, Calories
Prediction target	Calories burned (continuous value)
Task type	Regression
Value distribution	Calories burned spans roughly single digits up to ~300+ kcal per session, right-skewed rather than normal (published EDA on this

Item	Value
	dataset shows Body_Temp and Calories deviate from a normal distribution, unlike Age/Height/Weight/Duration). Exact min/max/mean must be confirmed by running df.describe() once the CSVs are loaded, and recorded here before the v1.0 model is signed off.
Primary evaluation metric	RMSE (Root Mean Squared Error) and R^2; MAE (Mean Absolute Error) tracked as a secondary, more interpretable metric
Why that metric	RMSE penalises large prediction errors in calories, which matters for user trust; R^2 shows overall fit quality; MAE gives an average "kcal off by" figure that is easy to communicate to a non-technical user
Level 0 baseline and its score	Predicting the training-set mean Calories value for every sample. By definition this scores $R^2 \approx 0$ and RMSE ≈ the standard deviation of Calories in the training split — to be computed and logged once the data is loaded.
Known data quality problems	Missing values, duplicate rows, and outlier heart-rate/duration entries; also a data-leakage risk — published EDA on this dataset found Weight and Duration correlate above 0.9 with each other, so one of the two should be dropped (or both retained only after re-validating the correlation on the actual loaded data) before finalising the feature set.
Level 1 baseline and its score	Plain Linear Regression on the scaled features: Validation MAE ≈ 18.0 kcal (Training MAE ≈ 17.9 kcal), based on a published reproduction of this exact dataset with a 90/10 train-validation split.
Target for this release	Beat the Level 1 linear-regression baseline by a wide margin using XGBoost/Random Forest: target Validation MAE ≤ 12 kcal and $R^2 \geq 0.97$ on the hold-out split, in line with published results on this dataset where XGBoost reaches a Validation MAE of ≈ 10 kcal (Random Forest ≈ 10.5 kcal) against Linear Regression's ≈ 18 kcal.

Dataset files used for development —
https://media.geeksforgeeks.org/wp-content/uploads/20240725130529/calories.csv

https://media.geeksforgeeks.org/wp-content/uploads/20240725130600/exercise.csv

Note: These are convenience mirrors of the same Kaggle "Calories Burnt Prediction" dataset used in several published tutorials and papers on this exact problem. Before final submission, replace this with the direct Kaggle dataset link and confirm the licence text on that page.

4. System Architecture

Login / Register → Dashboard → Today's Workout Input → Feature Engineer / Preprocessor → ML Prediction Engine → Calories Result → Save to Database → Updated History

Component	Responsibility
Auth Module	Handles user registration, login, and session management
Data Preprocessor	Cleans dataset, handles missing values, normalizes features
Feature Engineer	Prepares user-entered inputs into the model's expected feature format
Model Trainer	Trains supervised regression models (Linear Regression, Random Forest, XGBoost)
Prediction Engine	Generates the predicted calories-burned value for a submitted workout
History Manager	Stores and retrieves each user's past workout records
Web UI / Dashboard	Interactive interface for login, input entry, and viewing results/history

5. Functional Requirements

5.1 Authentication and Data Handling

FR-1.1 The system shall allow a new user to register with name, email, and password.

FR-1.2 The system shall allow a registered user to log in and log out securely.

FR-1.3 The system shall validate user-entered workout inputs and reject invalid values (e.g., negative duration, unrealistic heart rate) with an error message.

FR-1.4 The system shall normalize/scale numeric features before feeding them to the model.

5.2 Model-backed Requirements

FR-2.1 The system shall accept today's workout inputs (age, gender, height, weight, workout type, duration, heart rate, body temperature) from the user.

FR-2.2 The system shall send the submitted inputs to the ML calories-prediction model and return a predicted calories-burned value.

FR-2.3 The system shall display the predicted calories burned clearly on a dedicated result page.

FR-2.4 The system shall provide a fallback estimate (e.g., a simple MET-based formula) if the ML model service fails.

FR-2.5 The system shall display previous workout history including date, workout type, duration, heart rate, and calories burned.

FR-2.6 The system shall save today's workout inputs and predicted calories to the user's history/database upon prediction.

FR-2.7 The system shall refresh the dashboard and history view immediately after a new workout is saved.

FR-2.8 The system shall allow the user to edit or delete a previously saved workout record from their history — added because human correction of saved history was missing from the original list and is commonly required at review.

6. Non-Functional Requirements

NFR-1 The system shall return a calories prediction within 2 seconds of submission for a single workout entry.

NFR-2 The system shall use GitHub for version control.

NFR-3 The system shall log each prediction request with a unique ID and timestamp.

NFR-4 The system shall maintain 95% uptime during demo sessions.

NFR-5 The system shall support at least 50 concurrent users during demo testing.

NFR-6 The system shall use HTTPS for all client–server communication, store user passwords as salted bcrypt password hashes (never plain text or reversible encryption), and validate/sanitize every form input server-side (parameterised queries to prevent SQL injection, output escaping to prevent XSS) — added because a named security requirement was missing from the original list.

7. Build Plan

7.1 MVP

FR/NFR ID	Requirement	Why in MVP
FR-1.1, FR-1.2	Register and login	Without it, no user-specific history is possible

FR/NFR ID	Requirement	Why in MVP
FR-2.1, FR-2.2	Workout input and ML prediction	Core functionality
FR-2.3	Display result	Users need to see the outcome
FR-2.4	Fallback estimate	Guarantees a usable output if the model fails
FR-2.5, FR-2.6	History display and save	Enables the dashboard to show progress over time
NFR-1	Response time	Demonstrable performance
NFR-2	GitHub version control	Ensures collaboration

Deliberately not in the MVP: FR-2.8 (edit/delete workout history). This was cut because it needs a prior decision on whether an edited record should overwrite the original prediction or be re-submitted to the model and re-scored — and, if the model is later retrained, whether user-edited rows should be trusted as training data or excluded as potentially unreliable. That decision will be made in Sprint III once enough real usage data exists to judge how often users actually mis-enter workout details.

7.2 Build Order

Stage	Deliverable	Date
Development Sprint I	Prototype / initial working model	27 Sep 2026
Development Sprint II	50% functional application	17 Oct 2026
Development Sprint III	75–80% functional application	7 Nov 2026
Testing & Validation	Test report and bug log	17 Nov 2026
Deployment & Documentation	Deployed application, user manual, technical documentation	30 Nov 2026
Final Presentation	Final report, presentation and viva	5–10 Dec 2026

Stage / Date	What Will Be Working	FR/NFR IDs	How I Will Know It's Done
Sprint I — 27 Sep	MVP: register/login, workout input, ML prediction, fallback estimate	FR-1.1, FR-1.2, FR-2.1, FR-2.2, FR-2.4, NFR-1, NFR-2	A user can register, log in, submit a workout, and see a calories prediction returned in under 2 seconds; killing the model service manually still returns a labelled fallback estimate instead of an error.

Stage / Date	What Will Be Working	FR/NFR IDs	How I Will Know It's Done
Sprint II — 17 Oct	History display and save, input validation, feature scaling	FR-1.3, FR-1.4, FR-2.5, FR-2.6, FR-2.7	Submitting a workout immediately appears on the dashboard/history without a manual refresh; entering a negative duration or an out-of-range heart rate is rejected with a visible error message.
Sprint III — 7 Nov	Edit/delete history, prediction logging, named security controls, concurrency support	FR-2.8, NFR-3, NFR-5, NFR-6	A saved workout can be edited or deleted and the history updates accordingly; every prediction request appears in a log table with a unique ID and timestamp; passwords are verified to be stored as bcrypt hashes in the database; a load test (e.g. Locust) with 50 simulated concurrent users completes without failed requests.
Testing — 17 Nov	Functional test pass across all FRs, load test for NFR-5, uptime check for NFR-4, basic security testing (SQL injection / XSS attempts on all input fields), regression pass after the edit/delete feature	All FR/NFR IDs	A written test report lists each requirement with pass/fail status, plus a bug log with severity and resolution status for anything found.
Deployment — 30 Nov	Live deployment, user manual, technical documentation (architecture, API, DB schema)	All FR/NFR IDs	The application is reachable at a public URL, a PDF user manual and technical document are checked into the repository, and a fresh clone of the repo can be redeployed by following the documented steps.

8. Data Requirements

8.1 Entities

Entity: User

Field	Type	Constraint
user_id	UUID / INT	Primary key
name	String	Not null
email	String	Not null, unique
password_hash	String	Not null

Entity: Workout

Field	Type	Constraint
workout_id	UUID / INT	Primary key
user_id	UUID / INT	Foreign key → User
date	Date	Not null
workout_type	String	Not null
duration	Float	Not null, > 0
heart_rate	Float	Not null
body_temperature	Float	Optional
age, gender, height, weight	Mixed	Not null
predicted_calories	Float	Not null

9. Design Decisions and Alternatives Considered

#	Decision	Alternative Rejected	Reason
D1	Chose supervised regression	Unsupervised clustering of workouts	Dataset has a labeled continuous target (calories burned)
D2	Chose a relational (SQL) database	NoSQL document store	Workout data is tabular and consistent, and relationships (user → workouts) map naturally to foreign keys
D3	Added fallback MET-based estimate	No fallback, fail silently	Guarantees a usable result if the ML service is down

10. Acceptance Criteria

AC-1 Register/login with valid credentials → system grants access to the dashboard.

AC-2 Enter today's workout details → system returns a predicted calories-burned value within 2 seconds.

AC-3 Model unavailable → system falls back to a MET-based estimate and clearly labels it as an estimate.

AC-4 After a prediction is generated → system saves the workout and predicted calories to the database.

AC-5 After saving → the dashboard and history page immediately reflect the new entry.

11. Out of Scope for This Release

- Real-time biometric sensor integration (e.g., live wearable device streaming). Deferred to a later release.
- Personalized long-term fitness plan generation. Requires additional health data beyond this dataset's scope.
- Mobile app deployment. Web-only for now.